REMPLACE LES PDF JARGON · TES QUESTIONS ENTRE ## ## SONT RÉPONDUES ICI

Tout ce qu’il faut comprendre — Music, tests, Docs, Portainer, Maps

Ce fichier unique reprend, en français courant, les trois PDF du 6 octobre et les deux rapports Maps du 1er octobre. Chaque mot technique est expliqué. Tu peux relire d’une traite puis annoter.

> Ce PDF est ta priorité n°1 de ce soir. Le déploiement Hubera Docs sur le Nothing, Watchtower/CVE, etc. viennent **après** — tu pourras coller tes commentaires sur les propositions une fois Docs en ligne.

Sommaire

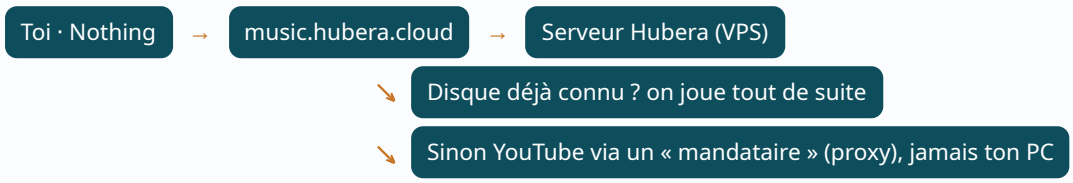
1. Comment lire ce cahier
2. Vocabulaire (à garder sous la main)
3. Qui parle à qui quand tu appuies Play
4. Tes questions sur le GET (auth, 206, DASH, ffmpeg, BUFFERING, Atlas, Android froid, Range, relais, googlevideo, 403)
5. Les 10 étapes du serveur, racontées comme une histoire
6. Pourquoi ce n’est pas ton PC · proxies · délais < 2 s
7. Ce que tu dois tester toi-même (ordre)
8. Hubera Docs + Portainer (sans casser quoi que ce soit)
9. Maps 1er octobre : guidage « comme Google » et départ / arrivée
10. Comment on travaille ensemble ensuite

Sources fusionnées : architecture-stream-proxy-music, tests-manuels-hubera, hubera-docs-portainer (6 oct) · Hubera-Maps-guidage-FPV et Hubera-Maps-depart-arrivee (1er oct ~15h58 et 16h26).

Comment lire

Les encadrés bleus = **ta question** (ce que tu as mis entre ##). Les paragraphes en dessous = la réponse en langage de tous les jours. Le vocabulaire de la page suivante sert de dictionnaire : si un mot revient, tu le retrouves là.

Imagine Hubera Music comme une radio. Ton téléphone n’appelle pas YouTube. Il appelle **notre serveur** (un ordinateur loué, le VPS). Le serveur va chercher le son, le range, et te le renvoie par petits paquets, comme Netflix qui n’envoie pas tout le film d’un coup.



Vocabulaire

Mot	Ça veut dire, concrètement
VPS	L'ordinateur distant où tourne Music (conteneur nommé encore « ytmusic »).
GET /api/stream/…	La demande du téléphone : « envoie-moi le son de ce titre ».
Auth / 401	On vérifie que tu es connecté à **Hubera** (pas à YouTube). 401 = « personne n'est connecté, je refuse ».
videoId	L'identifiant YouTube du clip, 11 caractères (ex. Flowers = BkSSOGNeWTM).
HTTP 206	Succès, mais **un morceau** du fichier seulement (normal, on lit par paquets).
Range	« Je veux les octets de X à Y » — comme demander les pages 1–10 d'un livre, puis 11–20.
Tête RAM / streamHeadCache	Le tout début du morceau (environ 1 Mo) déjà en mémoire vive du serveur, pour démarrer encore plus vite.
Disque / cache	Fichier déjà téléchargé sur le serveur : `.m4a` (son seul) ou `.mp4` (clip vidéo+son).
DASH	Découpage YouTube en tout petits segments « intelligents ». Notre lecteur téléphone n'aime pas ça. On veut un fichier d'un seul tenant.
Progressif	Un fichier continu, du début à la fin, comme un MP3 classique.
Muxé / itag 18 ou 22	Ancien format YouTube : image + son collés dans un seul `.mp4` (petite vidéo).
ffmpeg	Outil qui peut extraire uniquement le son d'une vidéo.
awaited / en fond	Attendre la fin avant de répondre au téléphone, ou continuer sans attendre.
BUFFERING	Le rond qui tourne, 0:00, pas de son. Le lecteur attend des données.
ExoPlayer	Le lecteur audio/vidéo d'Android dans l'app.
Proxy	Un intermédiaire : le serveur YouTube voit l'IP du proxy, pas celle du VPS (souvent bloquée).
googlevideo / GV	Les serveurs de fichiers de YouTube (le vrai fichier audio/vidéo).
Innertube / OAuth TV	La « API interne » YouTube, ici avec un compte TV connecté **sur le serveur**.
yt-dlp	Programme qui sait demander à YouTube l'adresse du fichier. `-g` = « donne-moi juste l'adresse, ne télécharge pas encore ».
403 / 404	Interdit / introuvable. Souvent : mauvaise IP, lien périmé, ou titre mort.
Titre chaud / froid	Chaud = déjà sur le disque du serveur. Froid = jamais (ou plus) en cache, il faut YouTube.
Android froid	Le téléphone Android demande un titre que le serveur n'a pas encore en fichier.
Relais maison	Option : passer par ton PC allumé. **Désactivé** aujourd'hui.
First-byte	Le temps avant le premier octet de son. Trop long = le lecteur tourne dans le vide.
OTA / apkAvailable	Mise à jour dans l'app. Disponible seulement si l'APK est **publiée** sur le serveur, pas juste parce que l'API a changé de numéro.
Overlay -r	Réinstaller l'app sur le téléphone sans tout casser (même signature).
Watchtower	Robot qui met à jour tout seul les conteneurs Docker. Dangereux pour Music si on a collé un correctif à la main.

Trois « connexions » distinctes

Compte	À quoi ça sert	Aujourd'hui
Hubera ID / Music	Savoir que c'est toi (biblio, likes, OTA). C'est le 401 du PDF.	Obligatoire pour écouter. Nothing = ton compte utilisateur, pas paul@ injecté.
Google YouTube **côté serveur**	Le VPS a le droit de demander des fichiers à YouTube (cookies / OAuth TV).	Déjà sur le volume du serveur. Ce n'est pas toi qui te connectes dans l'app.
Google de chaque utilisateur dans l'app	Lier « mon YouTube perso » à Hubera (playlists YouTube, likes YT…).	**Pas en place** comme SSO global. Décision produit plus tard (Hubera Hub).

> **Ta question.** Dois-je créer un Gmail dédié (pas paveldelhomme / pauldelhomme.pro) pour toutes les requêtes YouTube de tous les utilisateurs ?

Oui, c'est la bonne idée — mais pour le serveur, pas pour chaque téléphone. Les utilisateurs Hubera n'ont pas besoin d'un compte Google pour écouter. C'est le **VPS** qui parle à YouTube. Y coller ton Gmail perso, c'est coller ta vie privée à tous les titres demandés par tous les comptes. Un Google *technique* (créé par toi, uniquement pour Hubera Music serveur) est plus sain : YouTube Music / compte TV, cookies, OAuth. Un seul pour la machine. Tu le changes si YouTube le bloque, sans toucher à tes mails perso.

> **Ta question.** Sinon, chaque utilisateur devrait connecter son Google dans l'app, et ça devrait être global dans Hubera ?

C'est une **autre fonction** : « connecte ton YouTube pour retrouver tes playlists ». Ça n'est pas obligatoire pour que le son parte. Si on le fait un jour, oui : une seule connexion Google dans Hubera ID / Hubera Hub, réutilisée (Music, éventuellement d'autres). Ce n'est **pas** ce que fait le 401 du GET stream aujourd'hui. Le 401 dit seulement : « tu n'es pas loggé Hubera ».

Exception « relais interne » : un en-tête secret entre nos machines. Pas toi, pas l'utilisateur. Tu peux l'ignorer.

Le 206 n’est pas un échec

> **Ta question.** Si on a déjà envoyé le 1er Mo (206), ensuite il ne devrait plus y avoir de 206, on reconstruit la suite derrière ?

Si, il y aura encore des 206, et c’est voulu. Le 206 veut dire : « je t’envoie *cette tranche*, et je te dis la taille totale ». Ce n’est pas « fichier incomplet donc cassé ». C’est exactement Netflix / YouTube / Spotify : on n’attend pas d’avoir les 4 minutes en entier pour entendre la première seconde.

Exemple Flowers, fichier ~3,2 Mo sur le disque.

1. Le téléphone : « donne-moi les octets 0 à 524 287 » (les ~512 Ko du début). Serveur : **206**, voici ce bout, le tout fait 3,2 Mo.
2. Le lecteur démarre le son (souvent avant la fin de la phrase 1).
3. Le téléphone : « maintenant 524 288 à 1 048 575 ». Encore un **206**.
4. Et ainsi de suite jusqu’à la fin. Chaque 206 est un succès d’*une page* du livre.

La « tête RAM » : pour ~64 titres récemment vus, le tout début (1 Mo) est déjà en mémoire. Le premier 206 part sans ouvrir le disque ni YouTube. **La suite** vient du disque (ou de YouTube si le fichier n’existe pas). On ne s’arrête pas au 1er Mo : le lecteur enchaîne les Ranges.

Un vrai problème, ce serait un 206 qui ment sur la taille totale, ou un fichier qui ne grandit plus : là le lecteur croit que la chanson est finie au bout de 30 s. Ce n’est pas « trop de 206 ».

DASH, fichier muxé, ffmpeg — en clair

> **Ta question.** DASH = passer / bypasser ?

Non. DASH n'est pas « on saute une étape ». C'est un *format* YouTube : la chanson est découpée en micro-morceaux, avec un marqueur `ftyp=dash` au début du fichier. Notre app Android veut un fichier « normal » (un seul flux audio, du début à la fin). Si on lui donne du DASH, elle tourne dans le vide ou plante. Donc : si le fichier sent le DASH, on le jette et on redemande un fichier **progressif** (d'un seul tenant).

> **Ta question.** Muxé, itag 18/22 ?

YouTube numérote ses qualités (*itag*). 18 et 22 = vieille petite vidéo (souvent 360p/720p) où **l'image et le son sont dans le même fichier** `.mp4`. « Muxé » = multiplexé = collés ensemble. Comme une vidéo YouTube classique, pas comme un MP3. On peut quand même *écouter* : le lecteur Android sait prendre la piste son dans le MP4. C'est moins propre qu'un `.m4a` (son seul), mais ça joue.

> **Ta question.** ffmpeg extract en fond ? jamais awaited ? BUFFERING ? Flowers 3,2 Mo, ça devrait être combien ?

ffmpeg en fond : un petit robot sur le serveur ouvre le `.mp4` et en copie uniquement le son vers un `.m4a` pour la prochaine fois. Comme extraire la bande-son d'un film. « En fond » = on ne fait pas attendre le téléphone. « Jamais awaited sur le GET » = on n'attend pas que ffmpeg ait fini avant d'envoyer le premier son. Avant, on attendait : le téléphone restait bloqué même si le fichier était déjà là. C'était le bug Flowers.

BUFFERING, dans notre monde : l'écran « chargement », le temps à 0:00, pas de musique. Le lecteur a demandé le son et n'a pas assez d'octets (ou des octets du mauvais format) pour commencer. Ce n'est pas « la connexion est lente » forcément : ça peut être le serveur qui attend ffmpeg, YouTube, ou un relais mort.

3,2 Mo pour Flowers : ce n'est pas « trop petit pour une chanson ». Une chanson de ~3 min en bon AAC fait souvent 3 à 5 Mo *de son seul*. Ici 3,2 Mo c'est une *petite vidéo* (image + son). Ça suffit pour jouer. Le problème n'était pas la taille, c'était que le serveur **ignorait ce fichier** parce qu'il cherchait un `.m4a` qui n'existait pas, et retournait supplier YouTube. Il n'y a pas un « bon » nombre magique en Mo : le critère c'est « fichier assez gros, pas DASH, jouable ». Au-delà d'environ 512 Ko on considère que ce n'est plus une miette cassée.

Plusieurs videoId pour le même morceau — oui

> **Ta question.** On peut avoir une autre videoId du même morceau ? Lyric Video, etc. ?

Oui, et c'est fréquent. « Flowers » existe en clip officiel, lyric video, audio « Topic », live, reupload. Chaque page YouTube = un videoId différent. Notre « atlas » note : ce titre + cet artiste, on a déjà un fichier qui marche sous tel id. Si tu cliques une autre carte YouTube du même morceau, on peut rediriger (302 interne) vers le fichier déjà en cache au lieu de re-télécharger. Ce n'est pas « pirater une autre vidéo au hasard » : c'est coller les variantes d'un même enregistrement quand on les a reconnues.

> **Ta question.** Android froid — explique comme si je n'y connais rien.

Le téléphone Android demande un titre. Le serveur regarde son tiroir (le disque). **Vide** pour cet id = « froid » : personne (ou plus personne) n'a ce fichier sous la main. Alors on lance un téléchargement YouTube *du début du morceau seulement* (quelques centaines de Ko), on attend un tout petit instant (moins d'une seconde) au cas où ça commencerait à arriver, et on répond. On ne télécharge plus toute la bibliothèque ni 12 titres d'un coup : ça mettait YouTube en colère (cooldown = « tu abuses, j'arrête »).

« Prefix » = seulement le début, assez pour que le son parte. Le reste continue à arriver pendant que tu écoutes (le « disque qui grossit »).

Range, relais maison, format, erreurs GV

> **Ta question.** Range open-ended forcé en bytes=0-524287 ? Mid-range disque qui grossit ?

Parfois Android dit : « envoie-moi *tout* à partir de l'octet 0, sans dire où m'arrêter ». Ça s'appelle une requête *ouverte*. Le serveur peut alors rester coincé à préparer un flux énorme, et tu n'entends rien pendant 10–20 s. Alors on **traduit** ça en : « d'abord les 512 Ko du début » (0 à 524 287). Le son part. Ensuite Android demande tout seul la suite (un « mid-range » = un bout plus loin dans la chanson). Si le fichier n'est pas fini de télécharger, on sert ce qui est déjà là et on attend un peu qu'il allonge — « le disque qui grossit ».

> **Ta question.** Relais maison / ALLOW_STREAM_UPSTREAM / first-byte puis fallback VPS ?

Rassuré : ce n'est pas activé. Health : relais = aucun, autorisation = false. Ton PC n'est pas dans le chemin. Si on l'activait un jour : le serveur tenterait d'abord le PC (IP « box », souvent mieux vue par YouTube), mais seulement un court instant pour le *premier octet*. S'il ne vient pas, on bascule sur le VPS + proxies (« fallback »). Le relais mort d'avant ouvrait la porte et renvoyait du vide : BUFFERING infini. D'où le verrou : il faut un oui explicite dans Portainer.

> **Ta question.** Format + googlevideo — je n'ai rien compris.

YouTube ne nous donne pas un MP3 tout prêt. Il donne une **adresse temporaire** vers un gros fichier chez lui (googlevideo.com), valable pour **une** adresse Internet. Notre serveur, *en se faisant passer par un proxy*, 1) demande cette adresse, 2) télécharge le fichier **avec la même IP de proxy**, 3) te le redistribue. Si on envoyait l'adresse YouTube directement au Nothing, YouTube verrait l'IP du téléphone ≠ IP du proxy → refus (403). D'où : jamais de redirection du téléphone vers googlevideo.

> **Ta question.** 403/401/404 GV ? rotate proxy ? yt-dlp -g ? reject DASH → progressif ?

GV = googlevideo.com (les fichiers YouTube), pas « Google Vidéo » le produit. 401/403/404 = pas le droit / interdit / plus là. On jette l'adresse périmée, on change de proxy (rotate = on tourne), on redemande une adresse avec yt-dlp en mode « juste l'URL » (`-g`). Si le fichier reçu est du DASH, on le refuse et on insiste pour un fichier progressif.

Quand tu tapes un titre — ordre réel

```
Toi tu tapes Flowers
   1. Es-tu connecté Hubera ? Non → « refuse » (401). Oui → suite.
   2. C’est un MP3 que tu as importé toi-même ? On te le sert.
   3. On a déjà le début en mémoire (1 Mo) ? On l’envoie (206) pour que ça parte.
   4. On a un fichier .m4a ou un clip .mp4 assez gros, pas DASH ?
        Oui → on l’envoie tout de suite. En parallèle on extraie le son
        (ffmpeg) sans te faire attendre.
   5. On connaît une autre vidéo YouTube du même morceau déjà en fichier ?
        On bascule vers celle-là.
   6. Rien sur le disque (froid) : on commence à télécharger le début,
        on n’attend presque pas, on ne lance pas 40 titres d’un coup.
   7. Android a demandé « tout le fichier » ? On commence par 512 Ko.
   8. Relais PC : sauté (désactivé).
   9. Encore rien : on demande à YouTube l’adresse du fichier, on le
        télécharge via proxy, on te le transmet. Pas d’adresse YouTube
        brute vers le téléphone.
  10. YouTube dit non (403…) : autre proxy, autre adresse, jamais de DASH.
```

Le téléphone, pendant ce temps, n’a qu’une barre de chargement. S’il a des octets assez tôt → le son. Sinon → BUFFERING.

Ce n’est pas ton ordinateur de salon

Le trafic YouTube part du VPS, traverse des **proxies HTTP publics** (on en a une liste, on teste lesquels répondent, on note ceux qui arrivent vraiment à googlevideo). Ton IP box n’est pas utilisée. Le tunnel vers 172.17.0.1 est mort et ignoré.

Beaucoup de proxies « répondent au ping » mais YouTube les refuse : d’où des titres *jamais écoutés* encore lents. Les titres déjà en fichier (chaud, y compris .mp4) doivent partir en une fraction de seconde.

Pour viser < 2 s même à froid (plus tard)

- Garder disque d’abord (déjà).
- 10–30 proxies *résidentiels payants* (vraie box, pas datacenter), collés à YouTube le temps d’un titre.
- Après la 1re écoute, le fichier reste : la 2e est toujours rapide.
- Ne pas rallumer le PC. Ne pas envoyer googlevideo au téléphone. Ne pas lancer yt-dlp sur toute la biblio.

> Watchtower (mises à jour auto Docker) a été **arrêté** le 6 oct : il aurait pu remplacer le conteneur Music et perdre le correctif .mp4.

Ce que tu testes toi-même, dans l’ordre

Priorité : Music → Maps → Fuel. Puis Contacts, Mail, Photos, Calendar, Admin, Jobs, Pass, Drive, Tasks, Cook. DA / icônes : pas à juger maintenant.

Ne jamais reconnecter paul@ sur le Nothing. Overlay déjà fait Music 1.3.352 et Maps 0.1.72. Fuel 1.4.166 n’est pas encore sur les téléphones (souvent 1.4.165).

Music Nothing (15 min)

1. [ ] Compte : p+1.3.352, pas de « mise à jour obligatoire ».
2. [ ] 5 titres déjà écoutés : son en moins de ~2 s, pas de rond à 0:00.
3. [ ] 2 titres jamais écoutés : noter le délai. KO si > 15 s sans son.
4. [ ] Skip suivant × 4 : pas coincé sur « chargement de la musique qui suit ».
5. [ ] Pause, seek, paroles, un titre hors-ligne puis reco.

Maps 0.1.72

1. [ ] Carte, tuiles, croix de recherche seulement si le champ est vraiment ouvert + du texte.
2. [ ] Lieux sauvés. Véhicules Fuel visibles, véhicule actif par défaut.
3. [ ] Guidage : carte penchée, nom de rue, logo Fuel (pause / arrêter / plein). Croix = stop guidage et Fuel.
4. [ ] Départ au-dessus, arrivée en dessous, bouton inverser, noms sous les points (pas juste A/B).

Fuel

Un seul budget global visé (plus 7 cartes). Si tu vois encore 7 lignes, c’est l’ancienne APK — note-le. Un plein hors-ligne puis reco = un seul plein.

Smoke ensuite : Contacts, Mail, Photos (miniatures), Calendar (lieu + inviter plus tard), Pass, Drive, Tasks, Cook, Admin (pas de bouton qui force une OTA).

Hubera Docs + ce que tu vois dans Portainer

Hubera Docs = ton tableau de bord à toi (tâches, docs, PDF, avancement, retours). Ce n'est **pas** Hubera Admin. Auth Hubera ID, seulement tes mails. GitHub PavelDelhomme/HuberaDocs. Domaine visé docs.hubera.cloud (à brancher). Chaque produit a un DOCS.md qui pointe là.

Noté pour plus tard (pas ce sprint) : Contacts type Google, Calendar lieu+invitations, Hubera Hub, hors-ligne commun, mesh ADB, DA.

Nom Portainer	En vrai	Toi tu fais
ytmusic	Hubera Music	On garde. Volume intouchable.
gasoil-tracking	Hubera Fuel	On garde. Juste le nom est vieux.
cloudity	Encore les API Mail/Drive/Contacts…	**Ne pas éteindre. Les hubera-mail-web etc. sont surtout une vitrine nginx.**
hubera-cms-runtime	Le vrai CMS (pages, admin)	Normal à côté de hubera-cms.
cooking-recipes	Ancien Cook	On arrêtera après avoir vérifié Cook. On garde les disques.
budgetai	Budget, pas encore « hubera-budget »	Pas de suppression pour l'instant.
watchtower	Mises à jour Docker toutes les 5 min	Déjà stoppé (sauvait Music).
jobbingtrack-prod / preprod	Hubera Jobs	Prod = vrai. Preprod = labo (beaucoup de boîtes).

Compromis : une app = un Git + un disque, mais on assume deux couches le temps de migrer (vitrine Hubera + Cloudity). VTCBuilder garde le métier chauffeur. Press = sites / landings. Fuel reste une app ; Maps montre le garage, ça ne remplace pas Fuel.

Jamais « down -v », jamais renommer les volumes Music/Fuel.

Maps comme Google Maps — ce qui a été livré le 1er oct

Le PDF « guidage FPV » parlait de « première personne » : simplement **la carte penchée dans le sens de la route**, comme Google, pas vue du ciel. HUD = bandeau : distance, icône, nom de rue. « Puis » = la prochaine rue, phrase courte. Panneau 30 / zone 30. Logo Fuel en bas : pause, arrêter, plein. La croix arrête le guidage **et** le trajet Fuel (avant, Fuel continuait tout seul).

Villes et commerces (stations, pharmacies…) apparaissent selon le zoom. Un tap ouvre une fiche + itinéraire. Maison / travail plus petits.

Batterie : trois GPS possibles en même temps (Maps, Fuel, et Music en réseau). On a ralenti un peu Maps. On n'a pas coupé le GPS Fuel : les kilomètres seraient faux. Maps s'arrête écran éteint ; Fuel continue, c'est voulu.

Départ / arrivée (16h26) : plus des pins A/B anonymes. Champ du haut = départ (GPS → nom de rue, ou « Domicile »). Champ du bas = arrivée (Intermarché, etc.). Bouton inverser comme Google. Les points ont le nom en dessous. Versions de l'époque : Maps 0.1.64 puis 0.1.65 ; aujourd'hui tu es plus loin (0.1.72). L'idée est la même à retester.

OTA Maps : jamais forcée. Nothing souvent « à toi », pas overlay si tu t'en sers.

Après cette lecture

1. Relis surtout les pages 3 à 9 (vocabulaire + tes questions Music).
2. Fais les 15 min Music sur le Nothing (page 11).
3. Dès que Docs est en ligne : un retour par point (« je veux un Google serveur dédié », « Atlas OK », « DASH clair », etc.). Chaque retour = une tâche pour moi.

À faire après ce PDF (tu l'as demandé, en second) : Docs web + Nothing ; informer des mises à jour yt-dlp / YouTube / CVE ; ne pas rallumer Watchtower à l'aveugle sur Music.

> Compte Google *serveur* dédié : recommandé. Compte Google *dans l'app* pour chaque user : option Hub plus tard, pas nécessaire pour écouter. Relais maison : off. 206 : normal. DASH : format interdit, pas un bypass. Flowers 3,2 Mo : suffisant, le bug était de l'ignorer.